VISIONCARE

MÉTRICAS DE MANTENIBILIDAD Y EVOLUCIÓN

INFORME FINAL

Curso: Métricas y Estándares de Calidad

Integrantes:

Juan Diego Jiménez Horta

Aura María Cabrera Ardila

Docente: José Miguel Llanos Mosquera

Corporación Universitaria del Huila – CORHUILA

Fecha: 9 de noviembre de 2026

Repositorio: https://github.com/ItzJunixs/visioncare-metricas

2. Introducción

2.1 Tema y problema

VisionCare es una aplicación web para ópticas independientes. Cubre todo el recorrido del paciente hasta la entrega de sus gafas: registro de pacientes y fórmulas ópticas, inventario de monturas, órdenes de trabajo con su secuencia de estados y facturación con pagos parciales. La usan tres roles: administrador, vendedor/cajero y optómetra.

El problema que resuelve es concreto. Antes, historias, fórmulas, inventario, órdenes y pagos se llevaban en papel, hojas de cálculo y llamadas al laboratorio: los precios y el stock se desincronizaban entre copias, las fórmulas se transcribían dos veces y los pagos parciales se anotaban en un cuaderno.

VisionCare se desarrolló en el curso Complementaria III (versión 1.0.0, etiquetada en sus repositorios `vc-api`, `vc-portal`, `vc-db`, `vc-infra` y `vc-docs`). En este proyecto esa versión es el **objeto de medición**: no se reconstruye la aplicación, se evalúa su calidad interna.

2.2 Objetivo

Objetivo general. Medir la mantenibilidad de VisionCare v1.0.0 con métricas de mantenibilidad y evolución, interpretar los resultados frente a umbrales de referencia y proponer mejoras priorizadas y fundamentadas en la evidencia.

Objetivos específicos:

- Definir 10 requisitos funcionales y 3 no funcionales verificables, entre ellos un requisito de mantenibilidad medible.
- Construir un medidor reproducible en la carpeta de pruebas del backend y validarlo contra una herramienta externa.
- Aplicar cinco métricas: complejidad ciclomática, LOC, índice de mantenibilidad, suite CK y costo de mantenimiento por tipo.
- Identificar los problemas de mantenibilidad y proponer refactorizaciones con su técnica, prioridad y efecto esperado.

2.3 Alcance

Se mide todo el código de producción de la aplicación: el backend `vc-api` (Java 21, 146 archivos) y el frontend `vc-portal` (React + TypeScript, 23 archivos), más el historial de commits de los cinco repositorios para la métrica de costo. Las mejoras se **proponen** con su diseño y efecto esperado;

implementarlas queda fuera del alcance acordado, por lo que no se presenta comparación antes/después.

3. Requisitos

Los requisitos provienen del backlog de VisionCare (historias HU-01 a HU-10 y reglas de negocio BR-xx). Roles del sistema: **ADMIN**, **SELLER** (vendedor/cajero) y **OPTOMETRIST** (optómetra). El detalle completo está en `docs/requisitos.md`.

3.1 Requisitos funcionales

Tabla 1. Requisitos funcionales con criterio de aceptación y componente (trazabilidad).

ID	Descripción	Criterio de aceptación	Componente
RF-01	Registrar un paciente con tipo y número de documento, nombre, contacto y EPS (HU-01).	Con un documento nuevo, el paciente queda ACTIVE y aparece en la lista. Tipo + número de documento repetido se rechaza con 409 (BR-01).	pacientes · RegisterPatientUseCase
RF-02	Registrar una fórmula óptica: SPH, CYL, eje y ADD de OD y OI, y distancia pupilar (HU-02).	Un eje fuera de 0–180 se rechaza (BR-02). Una fórmula vigente reemplaza a la vigente anterior (BR-03).	pacientes · RegisterOpticalFormulaUseCase
RF-03	Buscar pacientes por nombre o documento (HU-03).	Devuelve las coincidencias; sin coincidencias devuelve lista vacía sin error.	pacientes · SearchPatientsUseCase
RF-04	Marcar el control periódico vencido (HU-04).	Si la fórmula vigente tiene más de 12 meses, el detalle muestra OVERDUE.	pacientes · GetPatientUseCase
RF-05	Registrar, editar y eliminar monturas (HU-05).	Precio de venta menor al de compra se rechaza (BR-04); SKU repetido se rechaza (BR-05); otro rol que no sea ADMIN recibe 403.	inventario · Register/Update/DeleteFrameUseCase
RF-06	Configurar el stock mínimo y alertar (HU-06).	Con stock igual o menor al mínimo, la montura se marca belowMinStock y se puede filtrar.	inventario · SetMinStockUseCase
RF-07	Aprobar una orden de trabajo descontando stock (HU-07).	Descuenta 1 unidad y registra el movimiento en la misma transacción (BR-07, BR-09); sin stock suficiente se rechaza y nada cambia (BR-06).	ordenes · ApproveWorkOrderUseCase

ID	Descripción	Criterio de aceptación	Componente
RF-08	Crear una orden de trabajo como cotización (HU-08).	Nace en QUOTE con código OT-AAAA-NNNN y copia de la fórmula vigente; sin fórmula vigente se rechaza.	ordenes · CreateWorkOrderUseCase
RF-09	Avanzar el estado de la orden (HU-09).	Solo Cotización → Aprobada → En laboratorio → Lista → Entregada, sin saltos (BR-08); cancelar exige motivo (BR-10); cada cambio queda en el historial (BR-11).	ordenes · AdvanceWorkOrderStatusUseCase
RF-10	Facturar la orden y registrar pagos (HU-10).	Al aprobar se emite FV-AAAA-NNNN automáticamente; un pago mayor al saldo se rechaza (BR-13); estado PENDING, PARTIAL o PAID según el saldo (BR-15).	facturacion · CreateInvoiceUseCase, RegisterPaymentUseCase

3.2 Requisitos no funcionales

Tabla 2. Requisitos no funcionales con descripción medible, verificación y estado en v1.0.0.

ID	Categoría	Descripción medible	Forma de verificación	Estado v1.0.0
RNF-01	Rendimiento	p95 < 300 ms con 50 usuarios concurrentes en la aprobación de órdenes y la búsqueda de pacientes.	Prueba de carga antes de cada release.	No verificado: el MVP no tiene prueba de carga.
RNF-02	Seguridad	100 % de los endpoints de escritura responden 401 sin token y 403 con un rol no autorizado.	FrameControllerTest y colección Bruno/Postman (casos 401/403).	Cumple (37/37 peticiones correctas).
RNF-03	Mantenibilidad	Ningún método o función supera una complejidad ciclomática V(G) de 10.	MetricasMantenibilidadTest (backend) y medir_frontend.py (frontend).	Cumple en backend (máx. 4); no cumple en frontend (7 funciones > 10, máx. 22).

Los tres RNF cubren tres características distintas de ISO/IEC 25010 (eficiencia de desempeño, seguridad y mantenibilidad). RNF-03 se propuso en este proyecto porque se puede verificar automáticamente con las métricas del curso.

4. Arquitectura

4.1 Estilo seleccionado

VisionCare es un **monolito modular en capas**: un solo desplegable (`vc-api`) dividido en cinco módulos de negocio (pacientes, inventario, ordenes, facturacion, usuarios) y un módulo `shared`. Cada módulo expone una fachada pública (`<Modulo>Facade`) y oculta su implementación en `internal/`, organizada en cuatro capas: presentación, aplicación, dominio y persistencia.

4.2 Justificación

Tabla 3. Justificación del estilo frente a las restricciones del proyecto (ADR-001).

Restricción	Cómo la atiende el monolito modular
Un desplegable, una base de datos, un pipeline	Un solo vc-api y un solo PostgreSQL: no hay red entre servicios ni consistencia eventual que operar.
Cinco contextos de negocio con vocabulario propio	Cada contexto es un módulo con frontera explícita: un cambio en facturación no toca pacientes.
Equipo académico sin persona dedicada a operaciones	Microservicios exigirían orquestar cinco despliegues; aquí basta docker compose up.
NFR-08: primera petición exitosa en menos de 1 hora	Un solo proyecto Maven y un solo docker compose.

Alternativas descartadas: capas planas sin módulos (con cinco contextos cualquier clase podría importar cualquier otra) y microservicios (costo operativo injustificado para el tamaño del equipo y del sistema).

Tabla 4. Otras decisiones de arquitectura.

ADR	Decisión	Efecto en la mantenibilidad
ADR-002	Un esquema de PostgreSQL por módulo, sin llaves foráneas entre esquemas.	La propiedad de cada tabla es visible; un módulo no puede hacer join con tablas de otro.
ADR-003	ordenes llama a InventarioFacade en la misma transacción (BR-09); la factura se crea por evento después del commit.	El descuento de stock es atómico y ordenes no conoce la facturación (bajo acoplamiento).
ADR-004	JWT sin estado; roles ADMIN, SELLER y OPTOMETRIST; 401 sin token y 403 sin rol.	La seguridad es transversal (un filtro), no se repite en cada caso de uso.

4.3 Diagramas

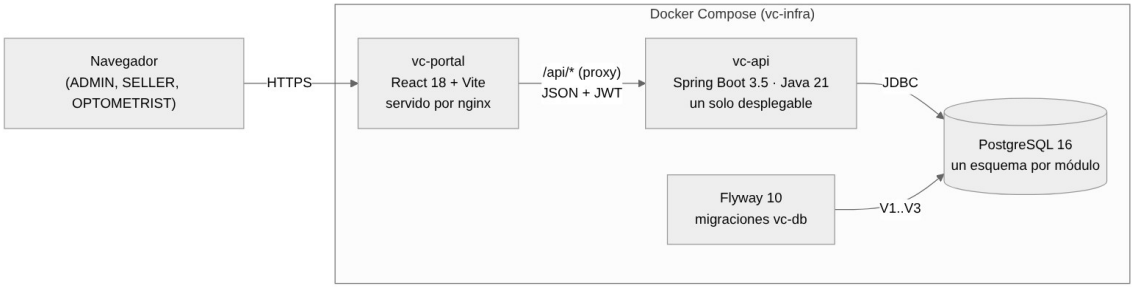


Figura 1. *Despliegue: el navegador habla con vc-portal (nginx), que reenvía /api a vc-api; Flyway aplica las migraciones de vc-db.*

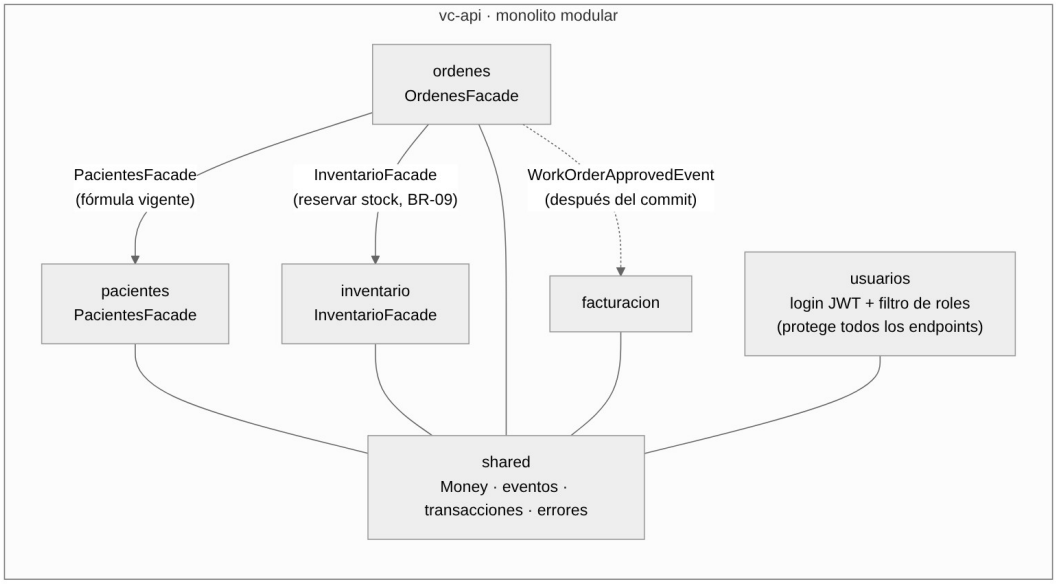


Figura 2. *Módulos y dependencias: solo tres dependencias entre módulos, todas por fachada pública o por evento. ArchUnit hace fallar el build si se importa el internal de otro módulo.*

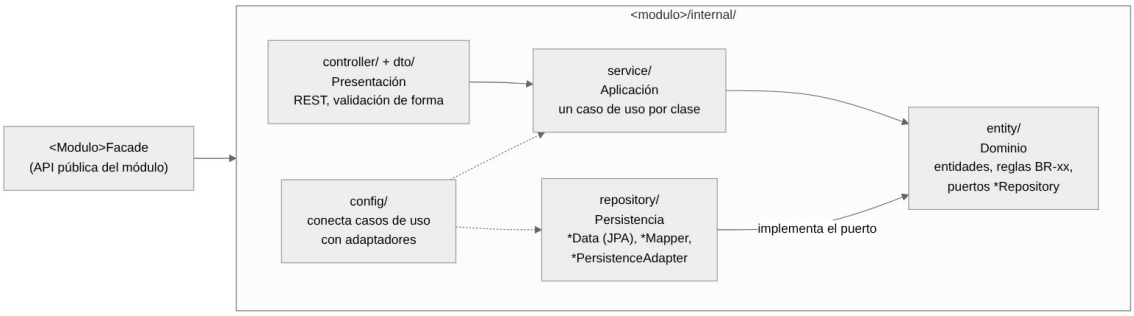


Figura 3. *Capas dentro de un módulo: las dependencias van hacia el dominio; repository implementa los puertos que declara entity; config conecta casos de uso y adaptadores.*

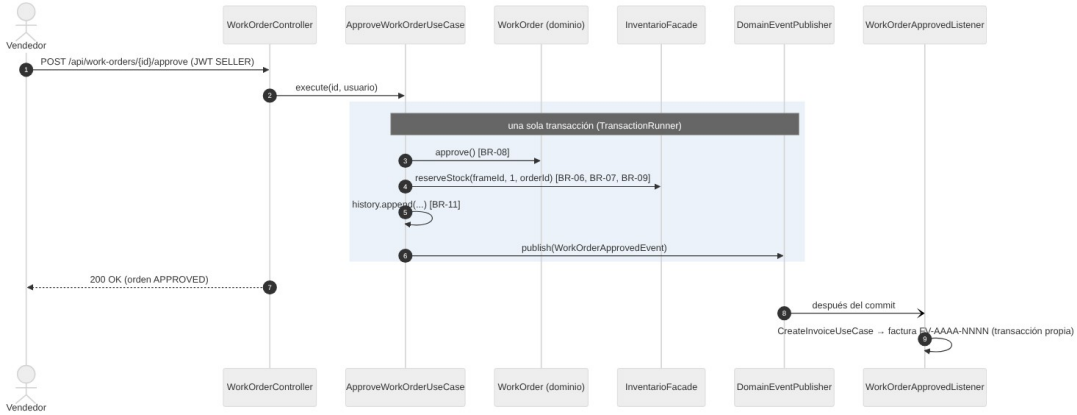


Figura 4. *Flujo principal (RF-07 y RF-10): aprobar una orden descuenta stock en una transacción y la factura se emite por evento después del commit.*

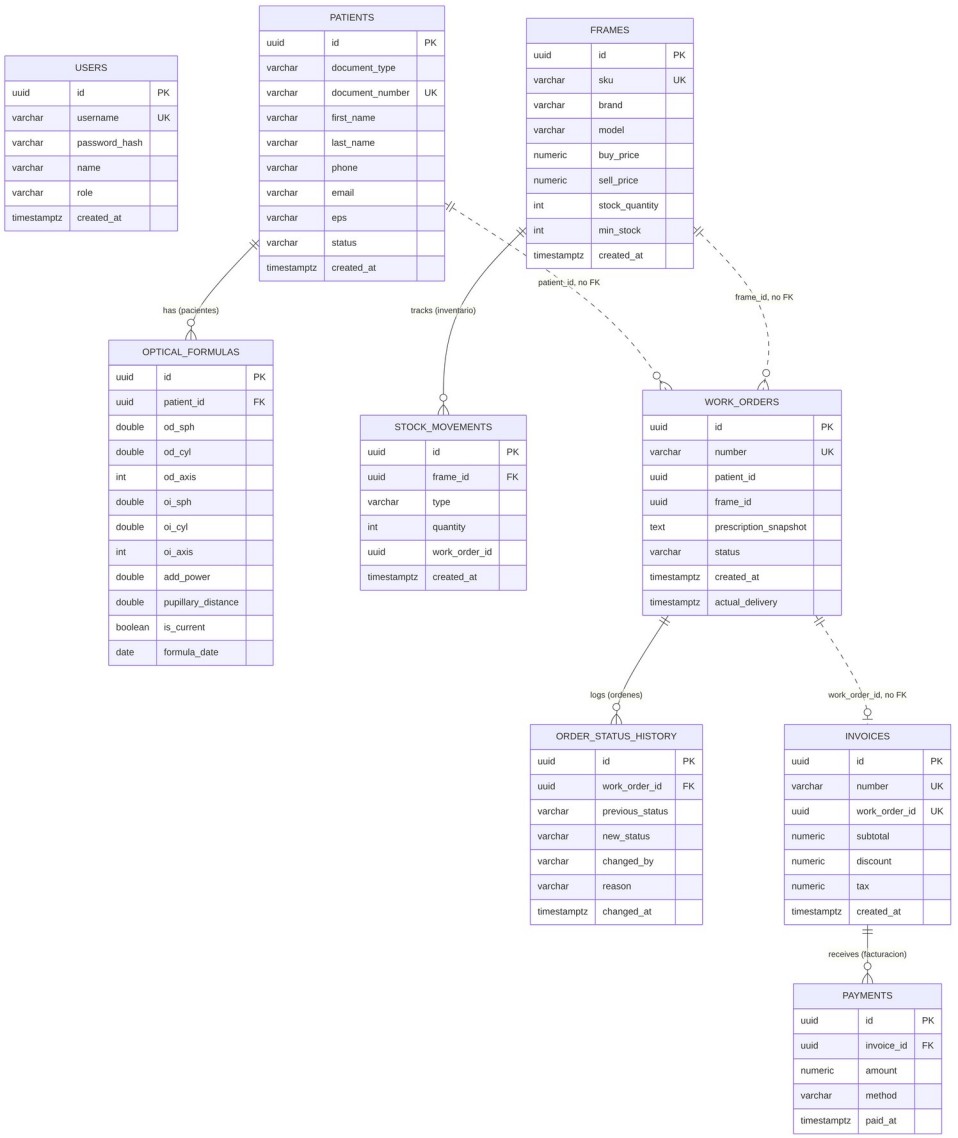


Figura 5. *Modelo de datos (vc-db): un esquema por módulo; las relaciones marcadas «no FK» cruzan esquemas y se validan por la fachada del módulo dueño.*

4.4 Estructura del proyecto

```
vc-api/src/main/java/co/visioncare/vcapi/
├── pacientes/
│   ├── PacientesFacade.java          <- API pública del módulo
│   └── internal/                     <- no se puede importar desde otro módulo
│       ├── controller/  dto/         <- presentación (REST)
│       ├── service/                  <- aplicación: un caso de uso por clase
│       ├── entity/                   <- dominio: entidades, reglas BR, puertos
│       ├── repository/               <- persistencia: JPA, mappers, adaptadores
│       └── config/                   <- conexión de dependencias
├── inventario/  ordenes/  facturacion/  usuarios/   (misma estructura)
└── shared/       (money, events, tx, exceptions, rest, config)

vc-portal/src/
├── api.ts  auth.tsx  useApi.ts  lib.tsx  toast.tsx  theme.tsx
├── components/   (Layout, Field)
└── screens/      (una pantalla por ruta: Patients, PatientDetail, Inventory, WorkOrderNew, ...)
```

En el repositorio del proyecto (`visioncare-metricas`) la aplicación se conserva en `vc-api/` y `vc-portal/` (cada una con su propio `src` y sus pruebas, porque son proyectos Maven y npm independientes); los medidores están en `vc-api/src/test/java/.../metricas/` y `metricas/`, y la documentación en `docs/`.

5. Implementación

5.1 Lenguaje y tecnologías

Tabla 5. Tecnologías utilizadas.

Parte	Tecnología
Backend	Java 21, Spring Boot 3.5.5, Spring Data JPA, Spring Security + JWT (jjwt 0.12), springdoc-openapi
Frontend	React 18, TypeScript 5.6, Vite 5, React Router 6
Base de datos	PostgreSQL 16 con migraciones Flyway 10 (vc-db)
Infraestructura	Docker Compose (vc-infra), nginx como proxy del portal

Parte	Tecnología
Pruebas	JUnit 5, ArchUnit, Spring Security Test, Vitest + Testing Library, Bruno/Postman
Medición	Medidor propio en Java (JUnit), lizard 1.25.1, Python 3.11+, matplotlib

5.2 Módulos

Tabla 6. Módulos de vc-api y requisitos que implementan.

Módulo	Responsabilidad	RF
pacientes	Pacientes, fórmulas ópticas y control periódico.	RF-01 a RF-04
inventario	Monturas, stock, movimientos y stock mínimo.	RF-05, RF-06
ordenes	Órdenes de trabajo, secuencia de estados, historial y aprobación con descuento de stock.	RF-07 a RF-09
facturacion	Facturas emitidas por evento, pagos parciales y estado según saldo.	RF-10
usuarios	Login JWT, contraseñas BCrypt y control de acceso por rol.	RNF-02
shared	Dinero, eventos de dominio, transacciones y manejo uniforme de errores.	Transversal

5.3 Instrucciones de ejecución

Aplicación. Con los repositorios `vc-api`, `vc-portal`, `vc-db` y `vc-infra` en la misma carpeta y Docker Desktop abierto:

```
cd vc-infra
docker compose up -d
# Portal: http://localhost:5173  ·  API: http://localhost:8080/api  ·  Swagger: /swagger-ui.html
# Usuarios de prueba: admin / Admin2026*  ·  seller / Seller2026*  ·  optometrist / Optom2026*
```

Medición (desde el repositorio `visioncare-metricas`; requiere JDK 21, Maven y Python 3.11+):

```
cd vc-api && mvn -q test -Dtest=MetricasMantenibilidadTest   # backend ->
vc-api/target/metricas/
cd ..
pip install -r metricas/requirements.txt
python metricas/medir_frontend.py             # frontend -> metricas/resultados/frontend-*
```

```
python metricas/validar_backend_con_lizard.py  # validación cruzada
python metricas/costo_mantenimiento.py         # costo por tipo de mantenimiento
python metricas/graficos.py                    # gráficos -> docs/metricas/graficos/
```

5.4 Pruebas

Tabla 7. Pruebas existentes y resultado.

Componente	Pruebas	Resultado
vc-api	14 pruebas unitarias + 7 reglas ArchUnit (fronteras entre módulos)	21 en verde
vc-portal	8 pruebas Vitest (cliente API y guardia de rutas)	Ejecutables con npm test
Integración	Colección Bruno/Postman: login, 401, 403, CRUD y reglas de negocio	37/37 peticiones correctas
Proyecto de métricas	MetricasMantenibilidadTest: 2 calibraciones, medición completa, RNF-03 y detección de God Class	5 en verde

5.5 Tabla de trazabilidad

Tabla 8. Trazabilidad de cada RF hasta el código, la interfaz, las pruebas y la evidencia.

RF	Endpoint principal	Caso de uso	Pantalla	Prueba automatizada
RF-01	POST /api/patients	RegisterPatientUseCase	PatientNew	Bruno/Postman
RF-02	POST /api/patients/{id}/optical-formulas	RegisterOpticalFormulaUseCase	PatientDetail	Bruno/Postman
RF-03	GET /api/patients?q=	SearchPatientsUseCase	Patients	Bruno/Postman
RF-04	GET /api/patients/{id}	GetPatientUseCase	PatientDetail	—
RF-05	POST/PUT/DELETE /api/frames	Register/Update/ DeleteFrameUseCase	Inventory, FrameNew	RegisterFrameUseCaseTest, DeleteFrameUseCaseTest, FrameControllerTest
RF-06	PATCH /api/frames/{id}/min-stock	SetMinStockUseCase	Inventory	Bruno/Postman
RF-07	POST /api/work-orders/{id}/approve	ApproveWorkOrderUseCase	WorkOrderDetail	ApproveWorkOrderUseCaseTest

RF	Endpoint principal	Caso de uso	Pantalla	Prueba automatizada
RF-08	POST /api/work-orders	CreateWorkOrderUseCase	WorkOrderNew	Bruno/Postman
RF-09	PATCH /api/work-orders/{id}/status	AdvanceWorkOrderStatusUseCase	WorkOrderDetail	Bruno/Postman
RF-10	POST /api/invoices/{id}/payments	CreateInvoiceUseCase, RegisterPaymentUseCase	Invoices, InvoiceDetail	Bruno/Postman

Solo RF-05 y RF-07 tienen pruebas unitarias; los demás se verifican con la colección HTTP, que requiere la aplicación levantada. Este hallazgo se retoma en el análisis (sección 8.5).

6. Definición de métricas

6.1 Selección y justificación

Se aplican **cinco métricas**, por encima del mínimo de tres de la guía. Dos son **estructurales** (V(G) y LOC) y tres evalúan el **diseño o la evolución** (índice de mantenibilidad, suite CK y costo de mantenimiento por tipo). La elección responde a las características de VisionCare:

- Sus requisitos se concentran en reglas de negocio (BR-02, BR-08, BR-09, BR-10). V(G) señala los métodos con más ramas, que es donde esas reglas se rompen al cambiar el código.
- Su arquitectura declara como objetivo bajo acoplamiento entre módulos y alta cohesión dentro de cada uno; CBO y LCOM de la suite CK lo verifican con números.
- Tiene dos lenguajes (Java y TypeScript): LOC, V(G) y MI se aplican a ambos, mientras que CK solo aplica al backend orientado a objetos.
- Es un sistema con historial en Git y Conventional Commits, lo que permite medir la evolución con el costo por tipo.

6.2 Medidor propio y su validación

Siguiendo los ejemplos en Java del módulo, se construyó un **medidor propio en la carpeta de pruebas del backend** (`vc-api/src/test/java/co/visioncare/vcapi/metricas/`). No modifica la aplicación: la prueba `MetricasMantenibilidadTest` lo ejecuta sobre `src/main/java` y deja los resultados en `target/metricas/`.

Tabla 9. Clases del medidor propio.

Clase	Función
AnalizadorLexico, Token	Tokeniza el código Java descartando comentarios y normalizando literales.
AnalizadorEstructura, ElementosCodigo	Reconoce clases, records, enums, campos, métodos y constructores.
MetricasLOC	LOC físicas, efectivas, de comentario y en blanco.
ComplejidadCiclomatica	V(G) = puntos de decisión + 1 y niveles de riesgo.
Halstead, IndiceMantenibilidad	Volumen de Halstead e índice de Oman y Hagemeister.
ClaseCK, SuiteCK	WMC, DIT, NOC, CBO, RFC, LCOM* y LCOM4.
RecomendadorRefactorizacion	Sugiere la técnica según los umbrales superados.
ReporteMetricas, MetricasMantenibilidadTest	Orquesta la medición, escribe los CSV y verifica el RNF-03.

Calibración. El medidor reproduce los ejemplos del material de clase: V(G) = 11 para `GestorPedidosAntes.procesar()` y 4 LOC físicas, 3 efectivas y 1 de comentario para `sumar()`. Ambos casos son pruebas automatizadas.

Validación cruzada. Se comparó método por método con lizard 1.25.1: coinciden V(G), NLOC y número de parámetros en **291 de 291** métodos comparables. El medidor propio detecta además **128 métodos** que lizard omite, porque lizard no reconoce clases con anotaciones con argumentos (`@RequestMapping("/api/...")`, `@Table(name = ...)`) ni métodos dentro de records: deja por fuera todos los controladores, las entidades JPA y los DTO. Con lizard solo no se habría detectado `PatientController`, la clase más acoplada del sistema.

El frontend se mide con lizard (`metricas/medir_frontend.py`), al que se agregan Halstead y MI con las mismas fórmulas del medidor Java.

6.3 Complejidad ciclomática V(G)

Tabla 10. Ficha de métrica: Complejidad ciclomática V(G).

Campo	Contenido
Nombre de la métrica	Complejidad ciclomática V(G) de McCabe (1976).
Atributo que mide	Complejidad lógica: caminos linealmente independientes de un método; mínimo de casos de prueba necesarios.

Campo	Contenido
Fórmula	Formal: V(G) = E − N + 2P. Práctica: V(G) = puntos de decisión + 1, contando if, for, while, case, catch, &&, || y el ternario ?: (en TSX también ?? y el renderizado condicional {x && <C/>}). else y default no suman.
Umbral e interpretación	1–10 bajo · 11–20 moderado · 21–50 alto · > 50 muy alto (tabla 2.3 del módulo). RNF-03: ningún método supera 10.
Herramienta	Backend: ComplejidadCiclomatica.java vía mvn -q test -Dtest=MetricasMantenibilidadTest. Frontend: lizard 1.25.1 vía python metricas/medir_frontend.py.
Alcance	vc-api/src/main/java (419 métodos y constructores) y vc-portal/src sin pruebas (193 funciones y componentes).
Justificación	Las reglas de negocio viven en métodos con ramas; V(G) dice dónde se concentra el riesgo y cuántas pruebas hacen falta.
Acción si se supera	Extract Method / Decompose Conditional; en React, Extract Component y Extract Custom Hook.

6.4 Líneas de código (LOC)

Tabla 11. Ficha de métrica: Líneas de código (LOC).

Campo	Contenido
Nombre de la métrica	LOC físicas, efectivas (NLOC), de comentario y en blanco; % de comentarios.
Atributo que mide	Tamaño del código y proporción de documentación interna.
Fórmula	Físicas = todas las líneas; efectivas = líneas con al menos un token de código; comentario = líneas solo con comentario; % comentarios = comentario / físicas × 100. NLOC por método = líneas efectivas entre la firma y la llave de cierre.
Umbral e interpretación	Más de 50 líneas efectivas por método es alerta (sección 2.1 del módulo). En React aplica al componente completo, JSX incluido.
Herramienta	MetricasLOC.java (backend) y metricas/comun.py → lineas_loc (frontend, reconoce {/ /} de JSX).
Alcance	Igual que V(G), por archivo y por método.
Justificación	NFR-08 pide que un desarrollador nuevo entienda y levante el sistema en menos de 1 hora; tamaño y documentación afectan esa curva de entrada.

Campo	Contenido
Acción si se supera	Extract Method / Extract Component; documentar clases de dominio y componentes principales.

6.5 Índice de mantenibilidad (MI)

Tabla 12. Ficha de métrica: Índice de mantenibilidad (MI).

Campo	Contenido
Nombre de la métrica	Índice de Mantenibilidad (Oman y Hagemeister).
Atributo que mide	Mantenibilidad global: combina tamaño léxico (Halstead), complejidad y LOC.
Fórmula	MI = 171 − 5,2·ln(HV) − 0,23·V(G) − 16,2·ln(LOC), con HV = N·log2(n). Por clase se usan los promedios por método (variante SEI).
Umbral e interpretación	≥ 85 alta · 65–85 moderada · < 65 baja. Valores mayores de 100 solo indican métodos triviales.
Herramienta	Halstead.java + IndiceMantenibilidad.java (backend); metricas/comun.py con las mismas fórmulas (frontend).
Alcance	Cada método o función; agregado por clase y por módulo.
Justificación	Resume en un número lo que V(G) y LOC dicen por separado y permite comparar backend con frontend.
Acción si se supera	Se actúa sobre el término dominante: LOC → Extract Method/Component; V(G) → Decompose Conditional.

6.6 Suite CK (Chidamber y Kemerer)

Tabla 13. Ficha de métrica: Suite CK (Chidamber y Kemerer).

Campo	Contenido
Nombre de la métrica	WMC, DIT, NOC, CBO, RFC y LCOM (con LCOM4 como confirmación).
Atributo que mide	Diseño orientado a objetos: complejidad ponderada, herencia, acoplamiento, respuesta y cohesión.

Campo	Contenido
Fórmula	WMC = Σ V(G) de los métodos. DIT = profundidad de herencia (Object = 1, padres del JDK con su profundidad real). NOC = subclases directas. CBO = tipos distintos usados, sin java.. *RFC = métodos propios + métodos distintos invocados. LCOM* (Henderson-Sellers) = (prom μ(Aj) − m)/(1 − m). LCOM4 = grupos de métodos conectados, sin accesores.
Umbral e interpretación	WMC > 20, DIT > 5, CBO > 10, RFC > 50, LCOM > 0,7. Criterio del equipo: LCOM se confirma solo si LCOM* > 0,7 y LCOM4 > 1. God Class = WMC, CBO y LCOM altos a la vez.
Herramienta	SuiteCK.java, ClaseCK.java y RecomendadorRefactorizacion.java (mismo test); salida clases-ck.csv.
Alcance	Las 154 clases, interfaces, enums y records de vc-api. No aplica a vc-portal (componentes funcionales, sin clases).
Justificación	El objetivo arquitectónico de VisionCare es bajo acoplamiento y alta cohesión; CBO y LCOM lo verifican.
Acción si se supera	WMC/LCOM → Extract Class / Move Method; CBO → Facade o dividir la clase; DIT → aplanar la jerarquía; más de 5 parámetros → Introduce Parameter Object.

6.7 Costo de mantenimiento por tipo

Tabla 14. Ficha de métrica: Costo de mantenimiento por tipo.

Campo	Contenido
Nombre de la métrica	Distribución del esfuerzo de mantenimiento por tipo (ISO/IEC 14764).
Atributo que mide	Evolución: en qué tipo de cambio se invierte el esfuerzo.
Fórmula	Cada commit se clasifica por su tipo de Conventional Commits: fix → correctivo; chore/build/ci → adaptativo; feat/perf/docs → perfectivo; refactor/test/style → preventivo. Se excluyen los commits iniciales. Costo ≈ número de commits y churn (líneas añadidas + eliminadas).
Umbral e interpretación	Se compara con Lientz y Swanson (1980): correctivo 21 %, adaptativo 25 %, perfectivo 50 %, preventivo 4 %.
Herramienta	python metricas/costo_mantenimiento.py (foto del historial en metricas/datos/) o --repos <rutas> para leerlos con git log.
Alcance	87 commits únicos de vc-api, vc-portal, vc-db, vc-infra y vc-docs.

Campo	Contenido
Justificación	Es la única métrica que mide la evolución y no solo el estado actual; sirve de línea base para etiquetar issues por tipo.
Acción	Registrar horas por issue etiquetado y aumentar la proporción de mantenimiento preventivo.

7. Resultados de las métricas

Medición sobre VisionCare v1.0.0. Los datos completos están en `metricas/resultados/` (CSV) y se regeneran con los comandos de la sección 5.3.

Tabla 15. Visión general de los resultados.

Indicador	vc-api (backend)	vc-portal (frontend)
Archivos medidos	146	23
Métodos o funciones	419	193
LOC efectivas / físicas	3 330 / 4 367	2 248 / 2 439
% de comentarios	3,4 %	0,5 %
V(G) promedio / máximo	1,21 / 4	2,47 / 22 (PatientDetail)
Funciones con V(G) > 10	0	7
Componentes con más de 50 NLOC	0	10
MI mínimo	80,4 (SecurityConfig.filterChain)	37,6 (PatientDetail)
Funciones con MI < 65	0	10

7.1 Complejidad ciclomática

Tabla 16. Distribución de V(G) por nivel de riesgo.

Riesgo	vc-api	vc-portal
Bajo (1–10)	419 (100 %)	186 (96,4 %)
Moderado (11–20)	0	6
Alto (21–50)	0	1
Muy alto (> 50)	0	0

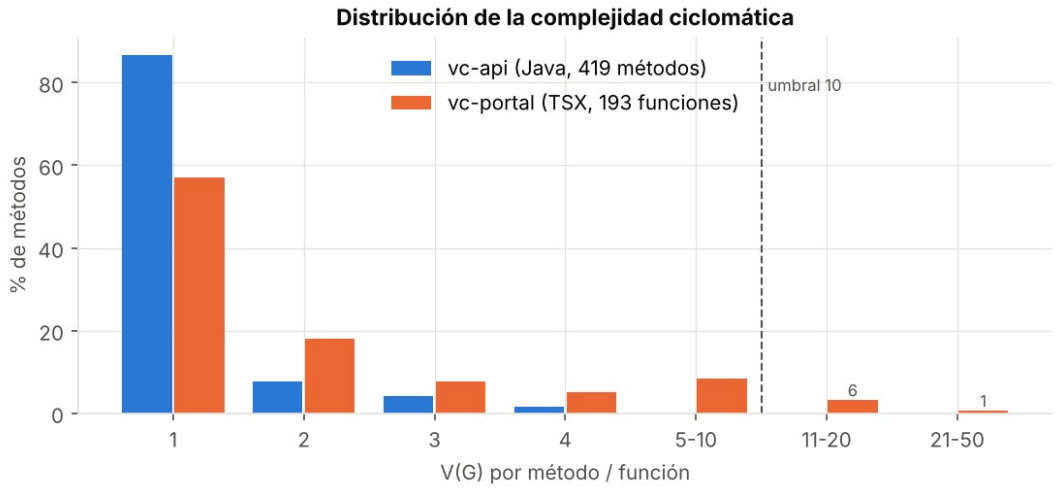


Figura 6. *Distribución de V(G) por método en backend y frontend.*

Tabla 17. Backend: los 10 métodos con mayor V(G).

Método	V(G)	NLOC	Parámetros	MI
Frame.register	4	14	7	95,3
PatientDetailResponse.from	4	12	1	94,7
Invoice.status	4	10	0	104,1
WorkOrder.advanceTo (BR-08)	4	10	1	103,3
WorkOrder.cancel (BR-10)	4	9	1	105,7
Document.Document	4	9	0	108,1
WorkOrderStatus.next	4	8	0	111,7
InvoicePersistenceAdapter.save	3	13	1	93,4
RegisterFrameUseCase.execute	3	13	1	95,1
JwtAuthenticationFilter.doFilterInternal	3	13	3	95,2

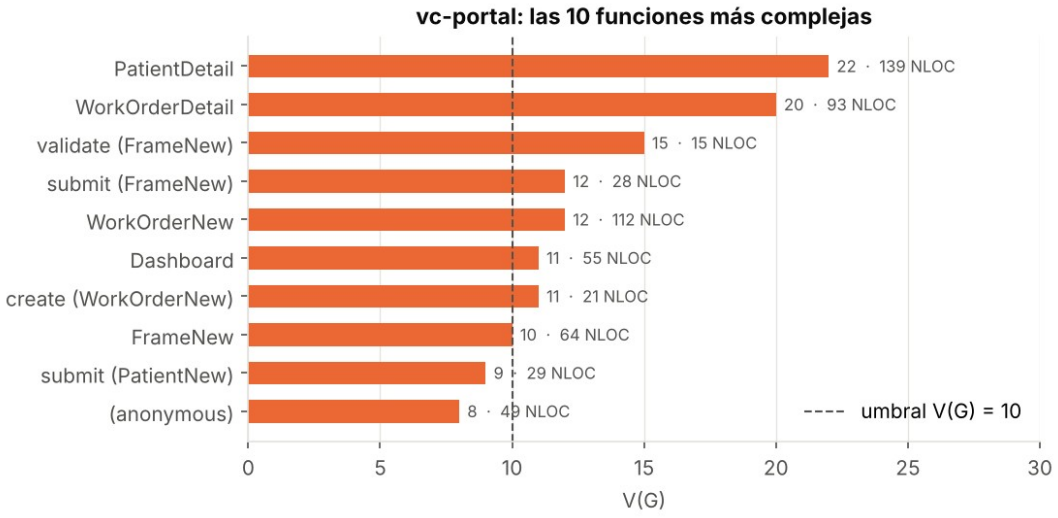


Figura 7. *Frontend: las 10 funciones con mayor V(G) frente al umbral de 10.*

7.2 Líneas de código

Tabla 18. LOC por módulo del backend.

Módulo vc-api	Físicas	Efectivas	Comentario	% comentarios
pacientes	1 211	940	30	2,5 %
ordenes	857	649	33	3,9 %
inventario	822	626	22	2,7 %
facturacion	650	491	25	3,8 %
usuarios	521	404	10	1,9 %
shared	293	211	28	9,6 %
Total	4 367	3 330	149	3,4 %

En el backend el método más largo tiene 26 NLOC (`SecurityConfig.filterChain`). En el frontend 10 componentes superan 50 NLOC: `PatientDetail` (139), `WorkOrderNew` (112), `WorkOrderDetail` (93), App (75, tabla de rutas declarativa), `PatientNew` (74), `InvoiceDetail` (68), `FrameNew` (64), `Dashboard` (55), `Inventory` (53) y `Patients` (51).

7.3 Índice de mantenibilidad

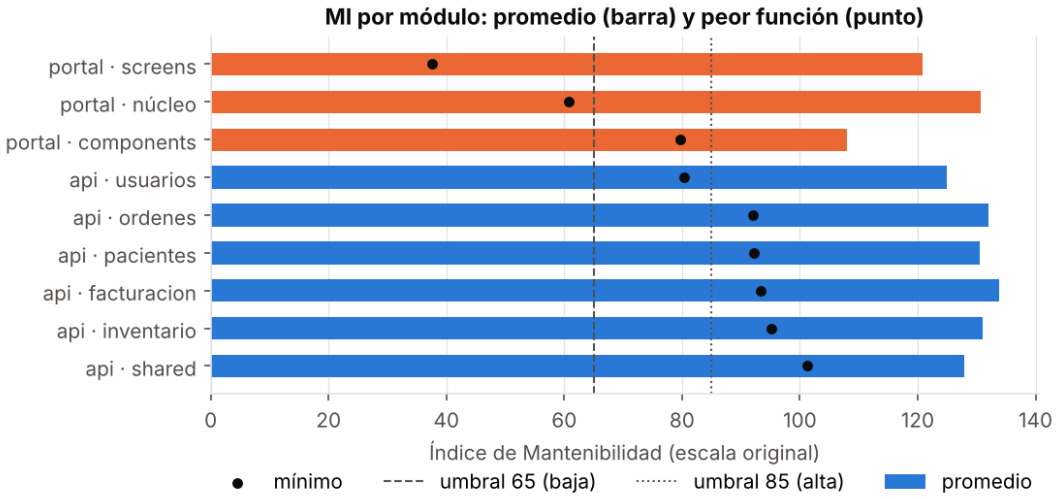


Figura 8. *MI por módulo: promedio (barra) y peor función (punto) frente a los umbrales 65 y 85.*

Tabla 19. Elementos con MI más bajo.

Elemento	MI	Nivel	Término dominante
PatientDetail (componente)	37,6	Baja	LOC 139 + V(G) 22
WorkOrderNew (componente)	44,9	Baja	LOC 112
WorkOrderDetail (componente)	48,7	Baja	LOC 93 + V(G) 20
InvoiceDetail	56,6	Baja	LOC 68
PatientNew	56,8	Baja	LOC 74
FrameNew	57,9	Baja	LOC 64
App	60,7	Baja	LOC 75 (declarativo)
Dashboard	62,9	Baja	LOC 55 + V(G) 11
Inventory	64,6	Baja	LOC 53
SecurityConfig.filterChain (peor del backend)	80,4	Moderada	LOC 26

7.4 Suite CK

Tabla 20. Resumen de la suite CK en vc-api (154 tipos).

Métrica	Promedio	Máximo	Clases sobre el umbral
WMC (> 20)	3,7	26 (Frame)	4: Frame 26, WorkOrder 22, Invoice 21, OpticalFormula 21
DIT (> 5)	—	6	9 excepciones de negocio (4 de los 6 niveles son del JDK)
NOC	—	4 (ConflictException)	Ninguna preocupante
CBO (> 10)	2,8	19 (PatientController)	6: PatientController 19, FrameController 12, WorkOrderController 11, OrdenesConfig 11, PacientesConfig 11, SecurityConfig 11
RFC (> 50)	—	46 (OpticalFormulaPersistenceAdapter)	0
LCOM* (> 0,7)	—	1,00	21, de las cuales solo 7 se confirman con LCOM4 > 1

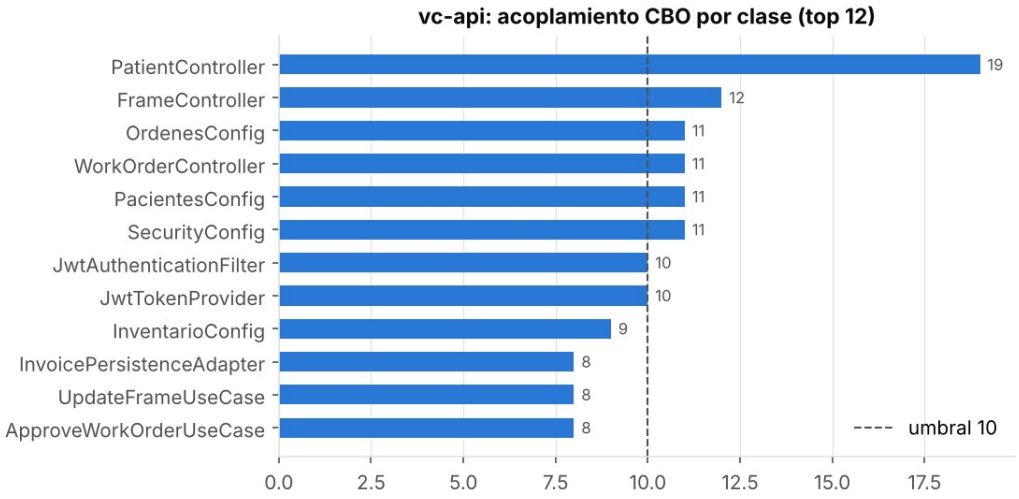


Figura 9. *Acoplamiento CBO por clase (las 12 más altas) frente al umbral de 10.*

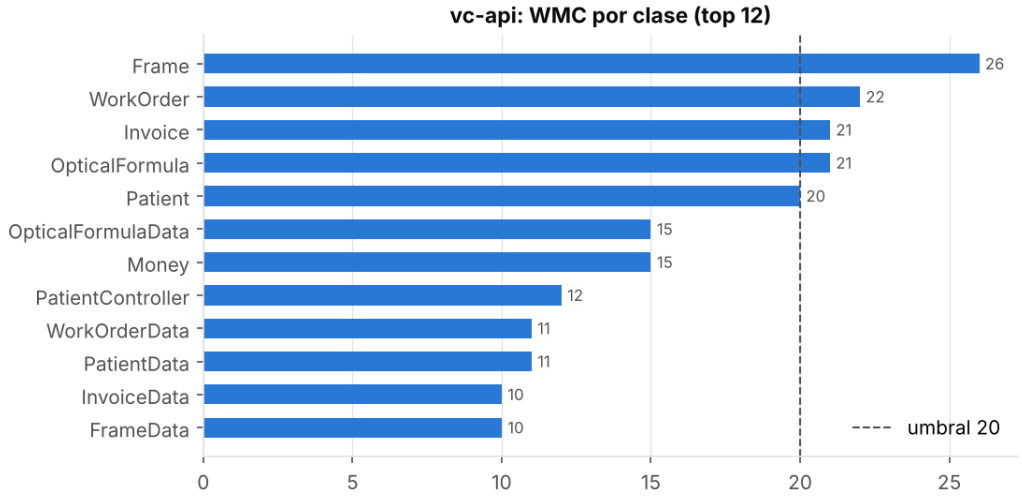


Figura 10. *WMC por clase (las 12 más altas) frente al umbral de 20.*

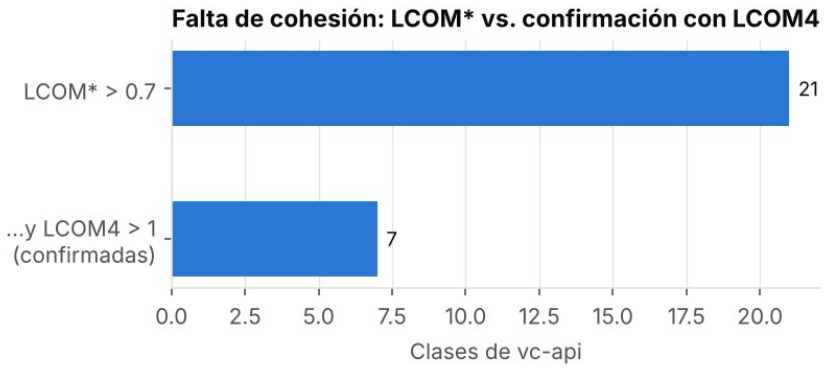


Figura 11. *Clases con LCOM* > 0,7 y cuántas confirma LCOM4.*

Además, **24 métodos** reciben más de 5 parámetros (dato complementario de *Long Parameter List*). El caso extremo es `OpticalFormula` con 12, porque esfera, cilindro y eje de cada ojo viajan como parámetros sueltos.

7.5 Costo de mantenimiento por tipo

Tabla 21. Distribución del mantenimiento por tipo frente a Lientz y Swanson.

Tipo	Commits (código)	%	Churn (líneas)	% churn	Todos los repos	Lientz y Swanson
Correctivo	3	7,9 %	654	5,2 %	15,4 %	21 %
Adaptativo	10	26,3 %	1 945	15,5 %	12,8 %	25 %
Perfectivo	22	57,9 %	7 844	62,6 %	67,9 %	50 %
Preventivo	3	7,9 %	2 092	16,7 %	3,8 %	4 %

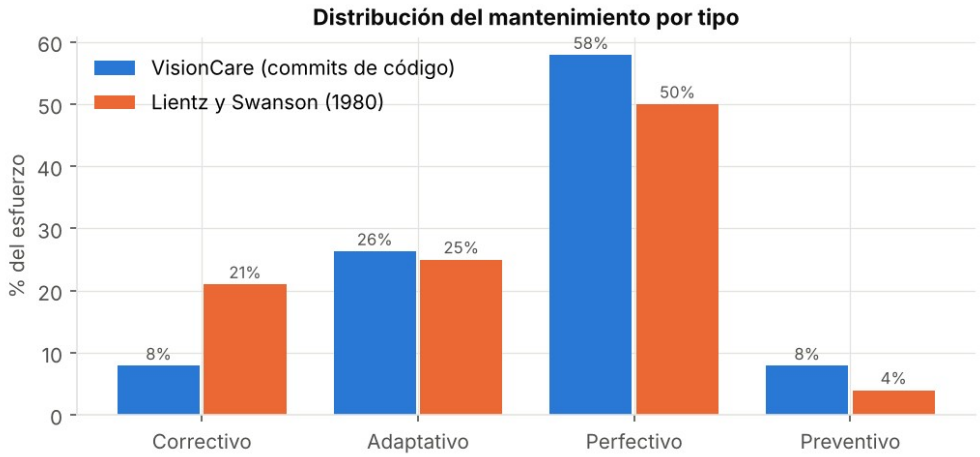


Figura 12. *Distribución del mantenimiento por tipo en los repositorios de código frente a Lientz y Swanson.*

Se analizaron 87 commits únicos, de los cuales 9 son de desarrollo inicial y se excluyen. El detalle por commit está en `metricas/resultados/commits-clasificados.csv`.

7.6 Evidencia de reproducibilidad

La medición se volvió a ejecutar en el equipo del grupo el 9 de noviembre de 2026: la prueba `MetricasMantenibilidadTest` terminó en verde y su `resumen.md` coincidió con los resultados guardados en el repositorio. La Figura 13 muestra la ejecución del script del frontend, con los mismos valores de las secciones 7.1 a 7.3.

```
C:\Users\juan2\Documents\Metricas y estandares de calidad\visioncare-metricas>python metricas/medir_frontend.py
# Resumen de metricas - vc-portal

Archivos: 23 | Funciones/componentes: 193

LOC fisicas 2439 | efectivas 2248 | comentario 12 (0.5%) | en blanco 179

## Distribucion de V(G)

| Riesgo | Funciones |
|---|---|
| Bajo | 186 |
| Moderado | 6 |
| Alto | 1 |
| Muy alto | 0 |

## Funciones que superan V(G) > 10 o NLOC > 50

| Funcion | Archivo | V(G) | NLOC | MI |
|---|---|---|---|---|
| PatientDetail | screens/PatientDetail.tsx | 22 | 139 | 37.6 |
| WorkOrderDetail | screens/WorkOrderDetail.tsx | 20 | 93 | 48.7 |
| validate | screens/FrameNew.tsx | 15 | 15 | 87.7 |
| WorkOrderNew | screens/WorkOrderNew.tsx | 12 | 112 | 44.9 |
| submit | screens/FrameNew.tsx | 12 | 28 | 76.3 |
| Dashboard | screens/Dashboard.tsx | 11 | 55 | 62.9 |
| create | screens/WorkOrderNew.tsx | 11 | 21 | 84.1 |
| FrameNew | screens/FrameNew.tsx | 10 | 64 | 57.9 |
| InvoiceDetail | screens/InvoiceDetail.tsx | 8 | 68 | 56.6 |
| Inventory | screens/Inventory.tsx | 7 | 53 | 64.6 |
| PatientNew | screens/PatientNew.tsx | 6 | 74 | 56.8 |
| Patients | screens/Patients.tsx | 4 | 51 | 66.6 |
| App | App.tsx | 1 | 75 | 60.7 |


C:\Users\juan2\Documents\Metricas y estandares de calidad\visioncare-metricas>
```

Figura 13. *Ejecución de python metricas/medir_frontend.py en el equipo del grupo.*

7.7 Tabla de resultados (Anexo C)

Tabla 22. Elementos fuera de umbral, estado y acción propuesta.

Elemento medido	Métrica	Valor obtenido	Umbral	Estado / acción propuesta
PatientController	CBO / LCOM* / LCOM4	19 / 1,00 / 9	≤ 10 / ≤ 0,7 / 1	Alto acoplamiento y baja cohesión: Extract Class (P2)
FrameController	CBO / LCOM4	12 / 6	≤ 10 / 1	Moderado: mismo patrón, revisar tras P2
WorkOrderController	CBO / LCOM4	11 / 4	≤ 10 / 1	En el límite: vigilar
OrdenesConfig, PacientesConfig, SecurityConfig	CBO	11	≤ 10	Excepción justificada: raíz de composición

Elemento medido	Métrica	Valor obtenido	Umbral	Estado / acción propuesta
Frame	WMC	26	≤ 20	Moderado: Extract Class de objetos de valor (P5)
OpticalFormula	WMC / LCOM4 / parámetros	21 / 2 / 12	≤ 20 / 1 / ≤ 5	Data Clump OD/OI: Introduce Parameter Object (P4)
WorkOrder, Invoice	WMC	22 / 21	≤ 20	Leve, LCOM4 = 1 (cohesivas): sin acción
GetWorkOrderUseCase, PacientesFacadeImpl, InvoiceController	LCOM4	2 / 2 / 3	1	Bajo: dos responsabilidades pequeñas; vigilar
9 excepciones de negocio	DIT	6	≤ 5	Falso positivo: jerarquía del JDK; sin acción
PatientDetail.tsx	V(G) / NLOC / MI	22 / 139 / 37,6	≤ 10 / ≤ 50 / ≥ 65	Alto: Extract Component + Custom Hook (P1)
WorkOrderDetail.tsx	V(G) / NLOC / MI	20 / 93 / 48,7	≤ 10 / ≤ 50 / ≥ 65	Moderado: Extract Component + hook de acciones (P1)
WorkOrderNew.tsx	V(G) / NLOC / MI	12 / 112 / 44,9	≤ 10 / ≤ 50 / ≥ 65	Moderado: un componente por paso (P1)
FrameNew.validate	V(G)	15	≤ 10	Moderado: validación por tabla de reglas (P3)
FrameNew.submit, WorkOrderNew.create, Dashboard	V(G)	12 / 11 / 11	≤ 10	Moderado: hook useApiAction (P3)
24 métodos del backend	Parámetros	6 a 12	≤ 5	Long Parameter List: Parameter Object (P4)

8. Análisis y plan de mejoras

8.1 El backend cumple; el frontend no

Backend. Los 419 métodos tienen V(G) ≤ 4 y NLOC ≤ 26, y ninguno baja de MI 80. Es consecuencia directa de la arquitectura (ADR-001): cada caso de uso es una clase pequeña con un único método

`execute`, y las reglas de negocio viven en entidades con métodos cortos. El RNF-03 se cumple en el backend y queda verificado automáticamente.

Frontend. Siete funciones superan V(G) 10 y diez componentes superan 50 NLOC con MI < 65. La causa es la misma en todos: **cada pantalla es un único componente que hace todo** (carga datos, guarda el estado del formulario, valida, llama a la API, maneja errores y dibuja varias secciones). `PatientDetail` mezcla además dos responsabilidades de negocio (ficha del paciente y CRUD de fórmulas), y por eso tiene el peor valor del sistema. El frontend no recibió la misma disciplina de separación de responsabilidades que el backend.

8.2 Duplicación en el frontend

- El bloque `setBusy(true); try { … toast(ok) … } catch (e) { toast(e instanceof ApiError ? … ) } finally { setBusy(false) }` se repite **8 veces** en 3 pantallas (PatientDetail, WorkOrderDetail, Inventory); cada copia suma +1 de V(G).
- `FrameNew.validate` encadena 7 `if` con 7 operadores lógicos (V(G) 15). Además BR-02 y BR-04 se validan en el frontend y también en el backend: si una regla cambia, hay que cambiarla en dos lugares.

8.3 Suite CK: problemas reales y falsos positivos

Tabla 23. Interpretación de los hallazgos de la suite CK.

Hallazgo	¿Problema real?	Razonamiento
PatientController: CBO 19, LCOM4 = 9, constructor con 9 parámetros	Sí	Expone dos recursos REST (/patients y /patients/{id}/optical-formulas) con 9 casos de uso inyectados; un cambio en fórmulas obliga a tocar el controlador de pacientes.
FrameController (CBO 12), WorkOrderController (CBO 11)	Moderado	Un recurso cada uno, con un caso de uso por endpoint; superan el umbral por poco.
21 clases con LCOM* > 0,7, solo 7 con LCOM4 > 1	No, para las otras 14	Entidades y clases JPA tienen un accesor por atributo; LCOM* lo interpreta como falta de cohesión. Sin LCOM4 se habrían propuesto 14 refactorizaciones innecesarias.
Frame (26), WorkOrder (22), Invoice (21), OpticalFormula (21) en WMC	Leve	El WMC viene de muchos métodos simples (V(G) promedio ≈ 1,2). WorkOrder e Invoice son cohesivas (LCOM4 = 1). Frame y OpticalFormula sí tienen un problema concreto (P4, P5).

Hallazgo	¿Problema real?	Razonamiento
Clases *Config con CBO 11	No (excepción justificada)	Son la raíz de composición de cada módulo; bajarles el CBO exigiría acoplar los casos de uso a Spring, en contra de la arquitectura en capas.
9 excepciones con DIT 6	No	4 de los 6 niveles son del JDK; la jerarquía propia es intencional: ApiExceptionHandler traduce a HTTP por el tipo base (409, 422, 404, 401).
24 métodos con más de 5 parámetros	Sí	Long Parameter List y Data Clump: odSph, odCyl, odAxis, oiSph, oiCyl y oiAxis viajan juntos por 6 firmas.

8.4 Costo de mantenimiento

El perfectivo (57,9 %) supera la referencia porque el sistema todavía agrega funcionalidad. El correctivo (7,9 %) es bajo porque VisionCare no ha estado en operación: los 3 `fix` se encontraron en pruebas. El adaptativo (26,3 %) es similar a la referencia (Docker, Compose, configuración). El preventivo (7,9 %) supera el 4 % en commits, pero su churn está inflado por la colección de Postman (1 537 líneas en un commit). **Limitación declarada:** no hay horas registradas y el historial de vc-api se consolidó en pocos días, así que el número de commits refleja cómo se organizó la entrega más que el esfuerzo real.

8.5 Pruebas: el riesgo que más condiciona las mejoras

vc-api tiene 14 pruebas unitarias y 7 reglas ArchUnit que cubren **3 de 25 casos de uso**; facturacion, pacientes y usuarios no tienen pruebas unitarias, y vc-portal no tiene pruebas de pantallas. El NFR-07 de VisionCare (cobertura ≥ 80 %) no se puede verificar porque no hay herramienta de cobertura. Como refactorizar es cambiar la estructura sin alterar el comportamiento observable (Fowler), sin pruebas no hay forma de demostrar lo segundo: por eso la red de pruebas (P6) es requisito previo de P1 y P2.

8.6 Plan de mejoras priorizado

Prioridad = riesgo del problema × beneficio ÷ esfuerzo. Los valores esperados son estimaciones que se confirmarían volviendo a medir.

Tabla 24. Propuestas de mejora priorizadas.

#	Problema	Elemento	Técnica	Riesgo	Esfuerzo	Métrica: antes → esperado
P6	Sin red de pruebas para refactorizar	facturacion, pacientes, pantallas del portal	Pruebas de caracterización + JaCoCo	Alto	10–12 h	Casos de uso con prueba 3/25 → 25/25; cobertura medible (NFR-07)
P1	Componente Dios / Long Method	PatientDetail, WorkOrderDetail, WorkOrderNew	Extract Component + Extract Custom Hook	Alto	8–10 h	V(G) máx. 22 → ≤ 10; NLOC 139 → ≤ 50; MI 37,6 → > 65
P3	Código duplicado y condicionales largos	8 bloques try/catch, FrameNew.validate	Extract Function (useApiAction) + tabla de reglas	Medio	3–4 h	V(G) validate 15 → ≤ 4; submit/create 11–12 → ≤ 5
P2	Alto acoplamiento, dos recursos en una clase	PatientController	Extract Class (OpticalFormulaController)	Medio	2 h	CBO 19 → ≈ 12 y ≈ 9; constructor 9 → 5 y 4 parámetros
P4	Data Clump / Long Parameter List	OpticalFormula	Introduce Parameter Object (EyePrescription)	Medio	3 h	Parámetros 12 → 8; WMC 21 → ≈ 18
P5	Clase con muchas responsabilidades	Frame	Extract Class (Pricing, StockLevel)	Bajo	3 h	WMC 26 → ≈ 16; parámetros 7 → 5
P7	Esfuerzo no medible	Proceso del equipo	Issues etiquetados por tipo con horas	Bajo	1 h	Costo por tipo en horas reales
P8	Métricas que se degradan sin aviso	Pipeline	Ejecutar los medidores en CI; fallar si V(G) > 10	Bajo	1–2 h	Prevención de regresiones

Orden recomendado: P6 → P1 → P3 → P2 → P4 → P5, con P7 y P8 en paralelo desde el inicio. **No se proponen cambios** (justificación en 8.3) para las clases *Config, la jerarquía de excepciones, WorkOrder e Invoice, ni para las 14 clases con LCOM* alto no confirmado.

8.7 Diseño de las propuestas principales

P1 · Dividir las pantallas del portal

`PatientDetail` pasa a orquestar componentes con una sola responsabilidad cada uno (boceto, no aplicado):

```
screens/PatientDetail.tsx              -> solo compone; V(G) esperado ≈ 4
├── components/PatientHeaderCard.tsx   -> nombre, documento, estado, botón Desactivar
├── components/CurrentFormulaCard.tsx  -> fórmula vigente y control periódico (HU-04)
├── components/FormulaHistoryTable.tsx -> historial con Editar / Eliminar
├── components/FormulaForm.tsx         -> formulario OD/OI (reutilizable en WorkOrderNew)
└── hooks/useFormulaForm.ts            -> estado, validate(), toPayload(), submit()
```

`WorkOrderDetail` se divide igual (`StatusTimeline`, `WorkOrderActions`, `useWorkOrderActions`) y `WorkOrderNew` pasa a un componente por paso del asistente.

P2 · Separar OpticalFormulaController

```
// Mismas rutas y mismo contrato OpenAPI: cero cambios para el portal
@RestController
@RequestMapping("/api/patients/{patientId}/optical-formulas")
public class OpticalFormulaController {
    public OpticalFormulaController(RegisterOpticalFormulaUseCase register,
ListOpticalFormulasUseCase list,
                                    UpdateOpticalFormulaUseCase update,
DeleteOpticalFormulaUseCase delete) { ... }
}
```

P3 · Hook useApiAction y validación por reglas

```
export function useApiAction() {                // elimina 8 copias del bloque try/catch/toast
  const toast = useToast();
  const [busy, setBusy] = useState(false);
  async function run(action, ok, fallback, after) {
    setBusy(true);
    try { await action(); toast(ok); after?.(); }
    catch (e) { toast(e instanceof ApiError ? e.message : fallback, "err"); }
    finally { setBusy(false); }
```

```
  }
  return { busy, run };
}
const reglasMontura = [                          // FrameNew.validate: de 7 if a una tabla
  { campo: "sku", falla: (f) => !f.sku.trim(), mensaje: "Ingresa el SKU (BR-05)." },
  { campo: "sellPrice", falla: (f) => Number(f.sellPrice) < Number(f.buyPrice), mensaje: "BR-
04 ..." },
];
```

P4 · EyePrescription como objeto de valor

```
// Los tres valores de cada ojo viajan juntos y BR-02 se valida en un solo lugar
public record EyePrescription(double sph, double cyl, int axis) {
    public EyePrescription {
        if (axis < 0 || axis > 180) throw new BusinessRuleException("Axis must be between 0 and
180");
    }
}
// OpticalFormula.register(patientId, EyePrescription od, EyePrescription oi, add, pd, current,
date)
```

Solo cambia el dominio interno: la tabla `pacientes.optical_formulas` y el contrato JSON se mantienen, y el mapeo se hace en `OpticalFormulaPersistenceAdapter`.

8.8 Verificación de las mejoras

Cada propuesta seguiría el ciclo **Medir → Detectar → Refactorizar → Verificar** del módulo: medición actual ya guardada en `metricas/resultados/`; detección contra umbrales (este capítulo); refactorización en ramas `refactor/<propuesta>` integradas por PR con revisión del otro integrante; y verificación volviendo a ejecutar los medidores con las pruebas de P6 en verde. Como las mejoras no se implementaron, no hay todavía tabla antes/después.

9. Conclusiones y lecciones aprendidas

9.1 Conclusiones

1. **La arquitectura se nota en las métricas.** El backend cumple con amplio margen los umbrales estructurales (V(G) máximo 4, MI mínimo 80,4) gracias a un caso de uso por clase, fachadas entre módulos y reglas ArchUnit.
2. **El riesgo de mantenimiento está en el frontend.** Siete funciones superan V(G) 10 y diez componentes tienen MI < 65; PatientDetail llega a V(G) 22 y MI 37,6 porque cada pantalla concentra todas sus responsabilidades.
3. **Un umbral sin interpretación engaña.** LCOM4 confirmó solo 7 de las 21 clases señaladas por LCOM*, y la DIT 6 de las excepciones viene casi toda del JDK.
4. **Sin pruebas no hay refactorización segura.** Solo 3 de 25 casos de uso tienen prueba unitaria; por eso la primera propuesta (P6) es la red de pruebas.
5. **La medición de costo describe un sistema en desarrollo.** 58 % perfectivo y 8 % correctivo; para conocer el costo real hay que registrar horas por issue.

9.2 Lecciones aprendidas

- Validar una herramienta contra otra y contra ejemplos conocidos antes de confiar en ella: lizard omitía controladores completos.
- Combinar métricas para confirmar un hallazgo: LCOM* con LCOM4, y V(G) con LOC dentro del MI.
- La arquitectura condiciona las métricas: el mismo equipo produjo un backend limpio y un frontend complejo porque solo el backend tenía una estructura que obligaba a separar responsabilidades.
- Las métricas orientan la discusión, pero la decisión de refactorizar necesita contexto de diseño, pruebas y revisión del código.

10. Aporte individual

El trabajo se organizó en un repositorio único con ramas por funcionalidad y pull requests: cada PR fue integrado a main por el otro integrante. El aporte de cada uno está respaldado por los commits y PR visibles en la pestaña *Contributors* del repositorio.

Tabla 25. Aporte individual respaldado por el historial del repositorio.

Integrante	Aporte	Respaldo en GitHub
Juan Diego Jiménez Horta (@ItzJunixs)	Proyecto base; medidor estructural (analizador léxico, LOC, V(G), Halstead, MI); medición del frontend y validación contra lizard; resultados; análisis del frontend y propuestas P1, P3 y P6; requisitos y arquitectura; integración del PR #6.	2 commits iniciales en main; autor de PR #1, #2, #3 y #7; merge de PR #4, #5 y #6
Aura María Cabrera Ardila (@aura200128cabrera-sys)	Suite CK y LCOM4; recomendador de refactorización; prueba JUnit de mantenibilidad y resultados del backend; costo de mantenimiento; gráficos; fichas de métricas; análisis CK y de costo con las propuestas P2, P4, P5, P7 y P8; README final; integración de PR; informe final.	Autora de PR #4, #5 y #6; merge de PR #1, #2, #3 y #7

10.1 Declaración de uso de inteligencia artificial

En el desarrollo de este proyecto se usó Claude (Anthropic) como asistente en las siguientes tareas: construcción de los medidores de métricas en Java (carpeta de pruebas de vc-api) y de los scripts en Python, generación de los gráficos y diagramas, apoyo en la organización del repositorio y redacción de borradores de la documentación, el informe y la presentación.

El equipo definió el alcance y las métricas a aplicar, revisó cada resultado y lo validó contra el código fuente y contra una herramienta externa (lizard, con coincidencia en 291 de 291 métodos), y tomó las decisiones de interpretación, como descartar las alertas de LCOM* no confirmadas por LCOM4. Ambos integrantes comprenden el código entregado y están en capacidad de explicarlo en la sustentación.

11. Referencias y repositorio

Repositorio: https://github.com/ItzJunixs/visioncare-metricas (rama `main`; versión entregada: etiqueta `v1.0.0`). Repositorios originales de VisionCare: github.com/code-corhuila/vc-api, vc-portal, vc-db, vc-infra y vc-docs (etiqueta v1.0.0).

Chidamber, S. R., y Kemerer, C. F. (1994). A Metrics Suite for Object Oriented Design. *IEEE Transactions on Software Engineering*, 20(6), 476–493.

Fowler, M. (2018). *Refactoring: Improving the Design of Existing Code* (2.ª ed.). Addison-Wesley.

Halstead, M. H. (1977). *Elements of Software Science*. Elsevier.

Henderson-Sellers, B. (1996). *Object-Oriented Metrics: Measures of Complexity*. Prentice Hall.

Hitz, M., y Montazeri, B. (1995). Measuring Coupling and Cohesion in Object-Oriented Systems. *Proceedings of the International Symposium on Applied Corporate Computing*.

ISO/IEC 14764:2006. *Software Engineering — Software Life Cycle Processes — Maintenance*.

ISO/IEC 25010:2011. *Systems and software Quality Requirements and Evaluation (SQuaRE)*.

Lientz, B. P., y Swanson, E. B. (1980). *Software Maintenance Management*. Addison-Wesley.

McCabe, T. J. (1976). A Complexity Measure. *IEEE Transactions on Software Engineering*, SE-2(4), 308–320.

Oman, P., y Hagemeister, J. (1992). Metrics for Assessing a Software System's Maintainability. *Proceedings of the Conference on Software Maintenance*, IEEE.

Corporación Universitaria del Huila. Módulo *Métricas de Mantenibilidad y Evolución del Software*, curso Métricas y Estándares de Calidad.

Yin, T. lizard: An extensible cyclomatic complexity analyzer (versión 1.25.1). https://github.com/terryyin/lizard